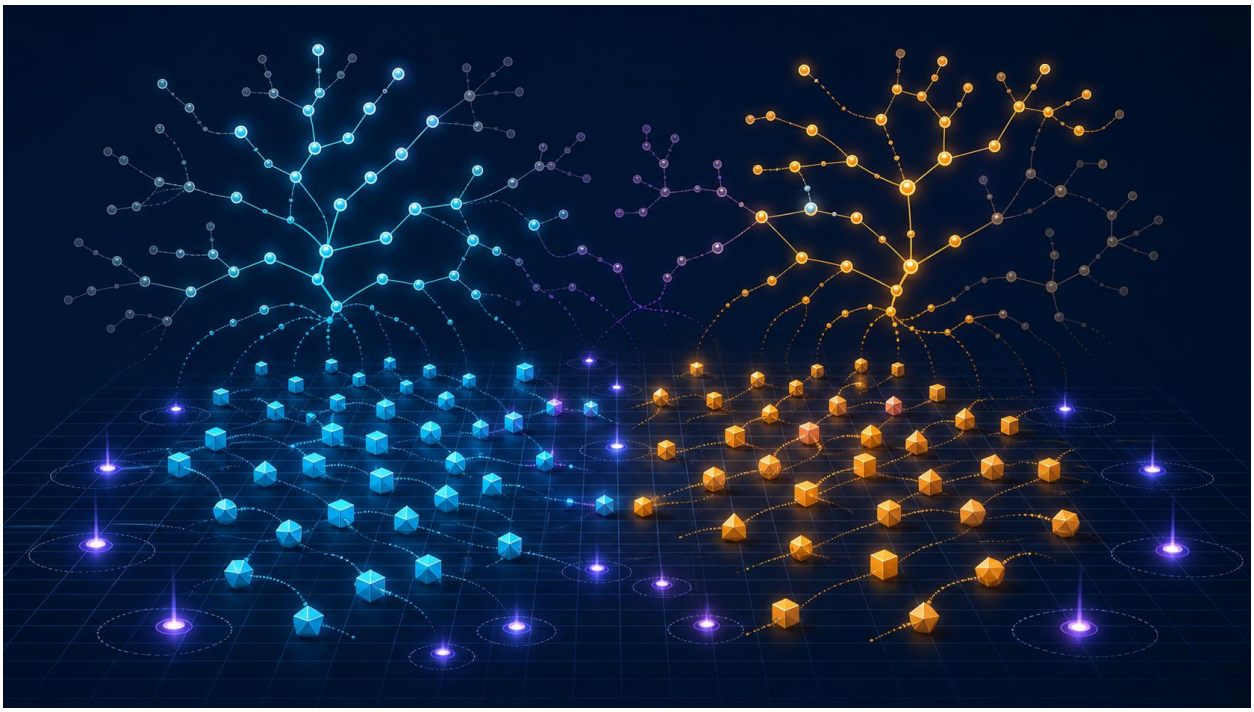
MASTER'S PROJECT PROPOSAL

ALIVE

Trait-Graph Evolution in Changing Worlds

A Spore-inspired simulation of modular adaptation under controlled environmental change



Concept visualization: a visual language for modular body plans and evolutionary pathways; no game assets are used.

CORE QUESTION **Does reciprocal evolution help populations maintain and generate more adaptable modular trait pathways when the world changes?**

Abstract

This project proposes a GPU-accelerated evolutionary simulation in which two populations of digital agents compete under finite resources while adapting to researcher-controlled changes in their environment. Agents inherit a modular, branching trait graph inspired by the functional part systems of creature-creation games: continuous traits determine the magnitude of capabilities, while graph modules determine the kinds of capabilities available. Unlike a game progression system, modules are not selected by a player; mutation proposes local changes and natural selection determines which body plans persist. The central experiment compares one-sided and reciprocal evolution under a preregistered schedule of resource regimes, including stable abundance, patchy scarcity, moving opportunities, and recovery. Outcomes quantify trait-graph depth, persistent branch diversity, historical novelty, adaptation lag, recovery when earlier worlds return, and cross-play against archived populations. The study tests sustained diversification and adaptation under explicit environmental variation, not unlimited open-ended evolution.

1. Research model

The project uses a visual, modular trait-graph model to make evolutionary pathways legible while retaining a rigorous fixed-shape GPU simulation. Spore is a design reference for readable functional parts, not a copied game system or scientific model.

Research question and hypothesis

QUESTION **Does reciprocal evolution produce more diverse, persistent, and adaptable trait pathways than one-sided evolution when environmental regimes change?**

- Reciprocal adaptation is expected to preserve or generate a wider range of viable trait combinations.
- Environmental transitions should make different modules useful at different times, preventing a single universally optimal build.
- A return to an earlier world will test whether populations retain, re-evolve, or replace previous specializations.

World and agent design

- A two-dimensional grid contains finite, regrowing resource values arranged according to the active world regime.
- Two populations share the grid. Agents move, consume resources, interact, expend energy, reproduce, mutate, and die.
- A fixed-capacity agent pool and alive mask maintain GPU-friendly JAX array shapes throughout births and deaths.
- The researcher changes the world on a predetermined schedule; agents do not alter terrain or resource rules.

Behavior policy

Each population uses one shared trait-conditioned policy. The policy receives local resource information, nearby agents, energy, and the agent's active-module representation. For the primary study, policies are

pretrained across all planned regimes and then frozen. This separates behavioral competence from evolutionary selection.

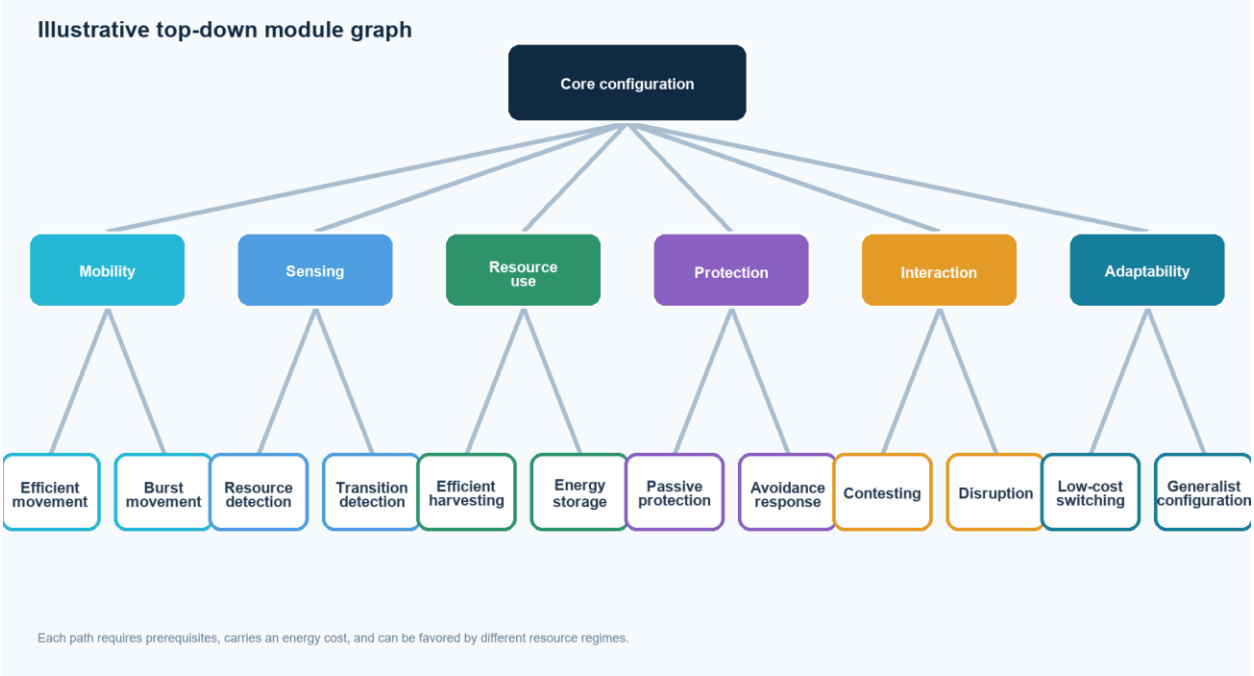
2. Trait architecture

The phenotype has two layers. Continuous traits determine how strongly an agent expresses a capability; graph modules determine what categories of capability it has. This avoids an unstructured list of dozens of independent values.

Continuous trait	Role	Typical trade-off
trait_0 - scale	Capability and action magnitude	Higher per-step metabolic cost
trait_1 - protection	Resistance to interaction pressure	Ongoing maintenance cost
trait_2 - intensity	Strength of resource contests	Greater energy expenditure
trait_3 - efficiency	Energy obtained per resource unit	Less flexibility or mobility
trait_4 - sensing	Range or quality of local information	Maintenance cost
trait_5 - storage	Energy retained across scarcity	Lower movement efficiency
trait_6 - flexibility	Behavior switching across regimes	Reduced peak specialization
trait_7 - recovery	Recovery following costly events	Higher baseline demand

Module graph

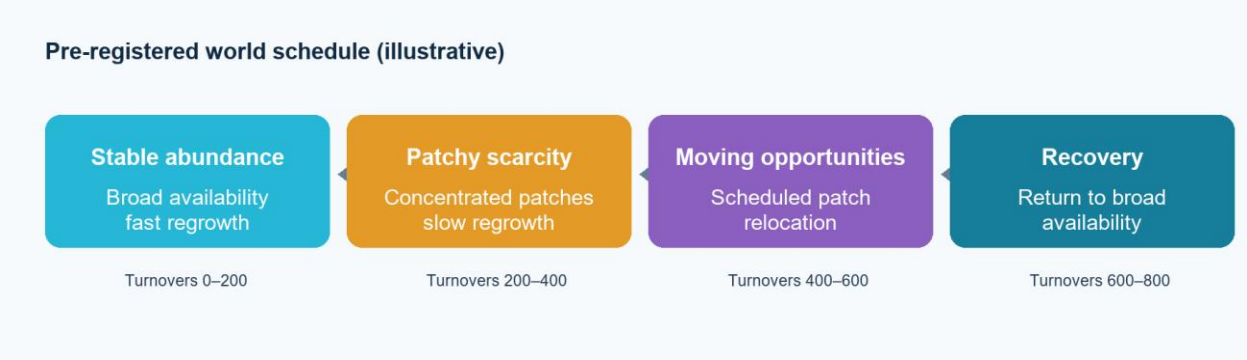
A graph of approximately 24-30 modules gives the project depth without turning it into an unconstrained invention system. Modules require prerequisites, incur energy costs, and can be beneficial only in particular resource regimes.



The graph is illustrative: final modules, prerequisites, incompatibilities, and costs are fixed after calibration and before confirmatory runs.

3. Controlled world change

The environment is an experimental treatment. Regime changes are defined before any confirmatory run and are identical across paired conditions. This prevents the researchers from tuning the world after observing which strategy wins.



Illustrative schedule only. Exact durations and parameters will be fixed after ecological calibration, then preregistered before confirmatory runs.

World regimes

Regime	Resource pattern	Trait pressure
Stable abundance	Broad availability; fast regrowth	Low-cost harvesting and simple configurations
Patchy scarcity	Concentrated patches; slow regrowth	Sensing, storage, efficiency, targeted movement
Moving opportunities	Resource patches relocate predictably	Mobility, transition sensing, flexible policies
Recovery	Return to broad availability	Retention, re-evolution, or replacement of strategies

Experimental conditions

Stage 1 establishes the static-world baseline. Stage 2 reuses the same evolutionary conditions under the world-change schedule. A-only and B-only are both required because the two populations are not assumed to be symmetric.

Condition	Population A	Population B	World
Frozen baseline	Frozen	Frozen	Stable
A-only baseline	Evolves	Frozen	Stable
B-only baseline	Frozen	Evolves	Stable
Reciprocal baseline	Evolves	Evolves	Stable
A-only changing	Evolves	Frozen	Scheduled changes

Condition	Population A	Population B	World
B-only changing	Frozen	Evolves	Scheduled changes
Reciprocal changing	**Evolves**	**Evolves**	**Scheduled changes**

4. Evaluation, implementation, and delivery

Primary measurements

- **Trait-graph depth:** How far persistent lineages move from the core configuration.
- **Persistent branch diversity:** How many distinct module combinations remain viable across turnover time.
- **Historical novelty:** Whether later successful configurations differ from all earlier successful configurations.
- **Adaptation lag:** How long population performance takes to stabilize after a regime transition.
- **Recovery response:** Whether previous specializations return, are retained, or are replaced after a familiar world returns.
- **Cross-play:** Whether later populations outperform archived populations under standardized environments, separating escalation from cycling.

Implementation phases

0	**Design lock**	Finalize module graph, regime equations, metric definitions, and success criteria.
1	**Core world**	Implement fixed-shape grid, energy economics, resource regrowth, births, and deaths.
2	**Trait graph**	Add inheritance, mutation, costs, prerequisites, and lineage tracking.
3	**Policies**	Pretrain and validate trait-conditioned policies across all planned regimes.
4	**Validation**	Benchmark throughput; validate novelty and cycling metrics using synthetic trajectories.
5	**Experiments**	Run static baselines, scheduled-change conditions, cross-play, and statistical analysis.

Time and compute planning

The expected project duration is 16-22 weeks. Early development should use scripted behavior and small worlds; the GPU benchmark determines the final scale. With frozen policies during evolutionary runs, the full project is estimated at roughly 100-250 GPU-hours: 20-60 for policy training and calibration, 25-70 for development experiments, 60-150 for replicated confirmatory runs, and under 10 for cross-play evaluation.

Scope boundaries and end goals

- Core scope: 20-30 modules, eight continuous traits, one resource type, three to four world regimes, two populations, and one fixed schedule.
- Out of primary scope: agent-created environmental modification, arbitrary trait invention, 3D body physics, and online learning during confirmatory evolutionary runs.
- Final deliverables: reproducible JAX simulator, benchmark, configs and seeds, trait-lineage visualizations, world-transition analyses, cross-play results, and a thesis-ready statistical report.

Pre-build decisions

- Lock the first module graph and its costs before policy pretraining; add modules only through versioned design changes.
- Choose the resource-regime schedule during calibration, then freeze it before confirmatory runs.
- Define the adaptation metric and persistence window using synthetic trajectories before examining main results.
- Treat a null result, convergence, cycling, or collapse as interpretable results rather than grounds for post-hoc world tuning.

DEFENSIBLE FINAL CLAIM **The project tests whether reciprocal evolution supports sustained modular diversification and faster adaptation under controlled environmental variation; it does not claim to prove unlimited open-ended evolution.**